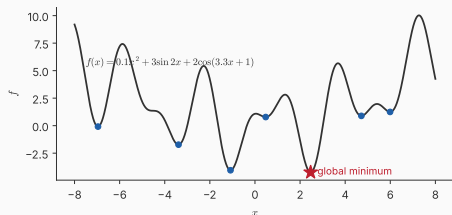


Optimization

5.2 Simulated annealing

José Firmino Aguilar Madeira

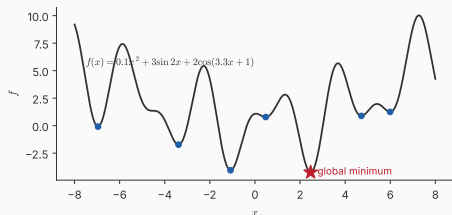
From GLODS to SA: what if we have only one point?



The chapter's reference function. From $x_0 = 6$, a local method (Chs. 2–4) ends at $x \approx 5.99$ ($f \approx 1.25$), not at $x^* = 2.47$: to get there one must **climb** two barriers (the higher one at $x \approx 3.68$).

5.1 (GLODS): several searches, to find other basins.

From GLODS to SA: what if we have only one point?

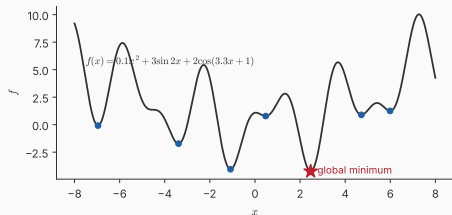


The chapter's reference function. From $x_0 = 6$, a local method (Chs. 2–4) ends at $x \approx 5.99$ ($f \approx 1.25$), not at $x^* = 2.47$: to get there one must **climb** two barriers (the higher one at $x \approx 3.68$).

5.1 (GLODS): several searches, to find other basins.

Question: what if we keep **only one point**, but allow it to **cross barriers**?

From GLODS to SA: what if we have only one point?



The chapter's reference function. From $x_0 = 6$, a local method (Chs. 2–4) ends at $x \approx 5.99$ ($f \approx 1.25$), not at $x^* = 2.47$: to get there one must **climb** two barriers (the higher one at $x \approx 3.68$).

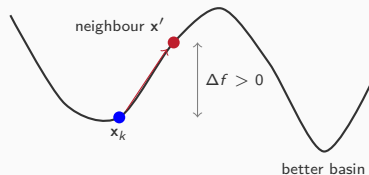
5.1 (GLODS): several searches, to find other basins.

Question: what if we keep **only one point**, but allow it to **cross barriers**?

The idea of simulated annealing

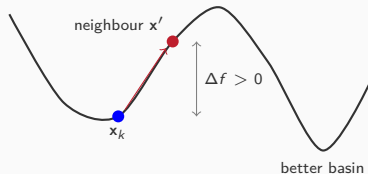
Leaving a local minimum requires **sometimes accepting uphill moves**.

Accept an uphill move? The Metropolis criterion



Question: the neighbour is worse. Do we accept it?

Accept an uphill move? The Metropolis criterion

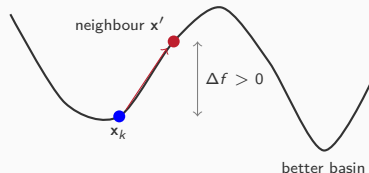


Question: the neighbour is worse. Do we accept it?

A local method: $\Delta f > 0 \Rightarrow$ **no**. Simulated annealing:

$\Delta f > 0 \Rightarrow$ **maybe**.

Accept an uphill move? The Metropolis criterion



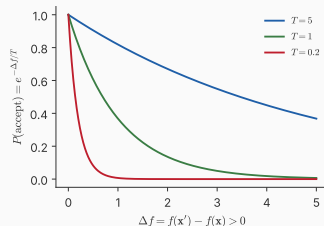
Question: the neighbour is worse. Do we accept it?

A local method: $\Delta f > 0 \Rightarrow$ **no**. Simulated annealing:

$\Delta f > 0 \Rightarrow$ **maybe**.

Metropolis criterion

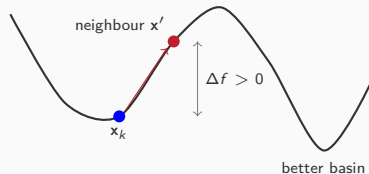
Always accept if $\Delta f = f(x') - f(x_k) \leq 0$; if $\Delta f > 0$, accept with probability $P(\text{accept}) = e^{-\Delta f/T}$.



The same uphill move $\Delta f = 1$: accepted with prob.
0.82 at $T = 5$, 0.37 at $T = 1$, 0.007 at $T = 0.2$.

T is not a physical temperature: it is a parameter on the scale of f .

Accept an uphill move? The Metropolis criterion

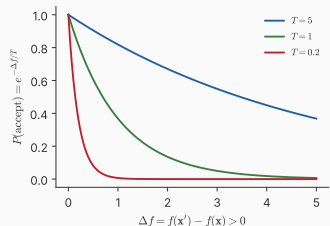


Question: the neighbour is worse. Do we accept it?

A local method: $\Delta f > 0 \Rightarrow$ **no**. Simulated annealing: $\Delta f > 0 \Rightarrow$ **maybe**.

Metropolis criterion

Always accept if $\Delta f = f(x') - f(x_k) \leq 0$; if $\Delta f > 0$, accept with probability $P(\text{accept}) = e^{-\Delta f/T}$.



The same uphill move $\Delta f = 1$: accepted with prob. 0.82 at $T = 5$, 0.37 at $T = 1$, 0.007 at $T = 0.2$.

T is not a physical temperature: it is a parameter on the scale of f .

Where the name comes from: a heated metal that is **cooled slowly** crystallizes in a minimum-energy state; cooled quickly, it ends up in a higher-energy state. Kirkpatrick, Gelatt and Vecchi (1983) carried the idea over to optimization with the Metropolis criterion (1953).

Hot → cold: exploration → refinement

High T

$P(\text{accept uphill}) \approx 1$: wanders between basins — **exploration**.



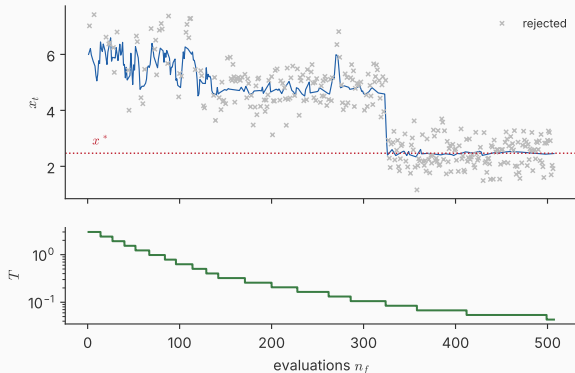
Intermediate T

accepts some uphill moves — **transition**: settles in a basin, with excursions.



Low T

$P(\text{accept uphill}) \approx 0$: only descends — **refinement**.



One run on the reference function: accepted (blue) and rejected (grey); below, T in steps, on a log scale.

exploration → refinement — the language of the chapter opening, in a single decreasing parameter.

Simulated annealing (as in class)

- Step 1** Choose $\mathbf{x}_0$, $T = T_0$ (high), n (accepted points per temperature), $\alpha \in [0.5, 0.99]$ and T_s ; set $t = 0$.
- Step 2** Generate a random neighbour $\mathbf{x}'$ of $\mathbf{x}_t$ (here $\mathbf{x}_t + \sigma\zeta$, $\zeta \sim N(\mathbf{0}, I)$, projected onto X).
- Step 3** $\Delta f = f(\mathbf{x}') - f(\mathbf{x}_t)$. If $\Delta f < 0$, accept: $\mathbf{x}_{t+1} = \mathbf{x}'$, $t = t + 1$. Otherwise, draw $r \sim U(0, 1)$: if $r < e^{-\Delta f/T}$, accept; otherwise, **go to Step 2**.
- Step 4** If $T < T_s$, **terminate**. Otherwise, if $t \bmod n = 0$, set $T = \alpha T$. Go to Step 2.

Deb (2012), Ch. 6.

t counts the **accepted** points; n_f counts all proposals. At low T almost everything is rejected and each step costs **more and more evaluations**.

in practice: also keep the best visited

$\mathbf{x}_{\text{best}}$

The algorithm returns the final point $\mathbf{x}_t$, which may have got worse *on purpose*. Only values of f .

Discrete variables

$x_i \in \{1, \dots, 5\}$ (thickness, type, component): change a random variable to the adjacent category, $(2, 4, 1, 3) \rightarrow (2, 4, 2, 3)$. Permutations: swap two positions, $(1, 2, 3, 4, 5) \rightarrow (1, 4, 3, 2, 5)$. **No continuous distance and no derivatives: it is enough to know how to generate a neighbour.**

How fast to cool? Practice and theory

Too fast

T drops before the point has explored: it quickly behaves like a local method and stays in the first basin.

Too slow

Explores better, but costs many evaluations: the cold steps are the most expensive.

In class: steps of n accepted points, then $T = \alpha T$.

Variant: $T = c T$ at every proposal, with a budget $N_{\max}$.

Geman, S., Geman, D., IEEE Trans. Pattern Anal. Mach. Intell. 6(6):721–741, 1984.

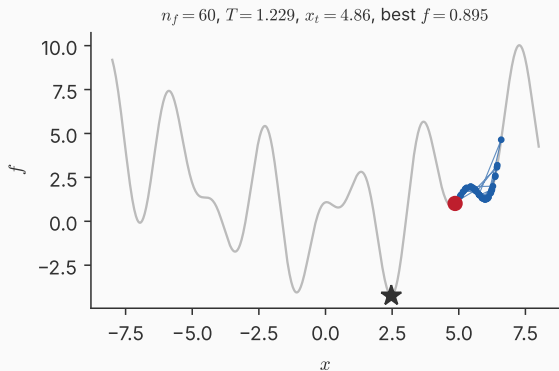
Hajek, B., *Cooling schedules for optimal annealing*, Math. Oper. Res. 13(2):311–329, 1988.

Theory (Geman & Geman, 1984; Hajek, 1988)

In a finite space, with logarithmic cooling $T_k = d / \ln(1 + k)$, $k \geq 1$, and $d \geq d^*$ (depth of the deepest local, non-global minimum), the probability of being at a global optimum tends to 1.

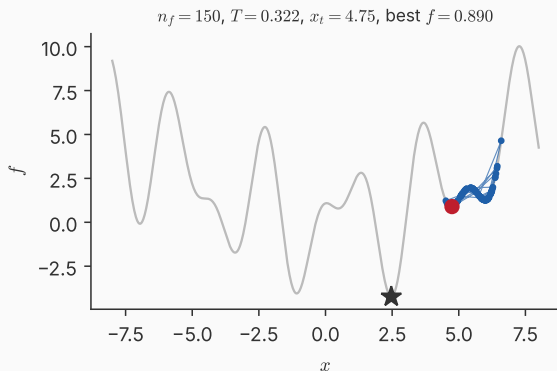
The theoretical guarantee requires cooling too slow to be normally used in practice. Practical (geometric) schedules do not have this guarantee (the best visited point has at least the guarantee of random search, 3.2.1 — equally slow).

One run step by step: $x_0 = 6$, $T_0 = 3$, $\alpha = 0.8$, $n = 10$, $T_s = 0.05$, $\sigma = 0.5$



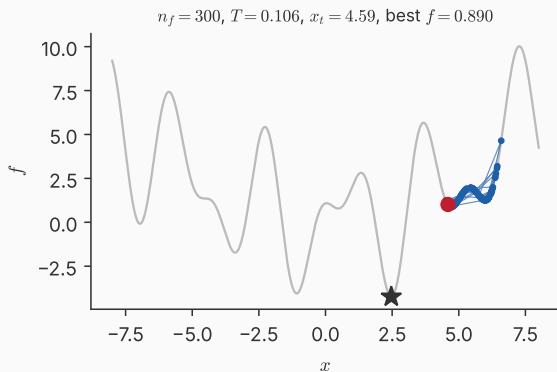
- $n_f = 60$, $T \approx 1.2$: already in the basin of 4.7 (local minimum, $f = 0.89$), but goes up and down a lot.

One run step by step: $x_0 = 6$, $T_0 = 3$, $\alpha = 0.8$, $n = 10$, $T_s = 0.05$, $\sigma = 0.5$



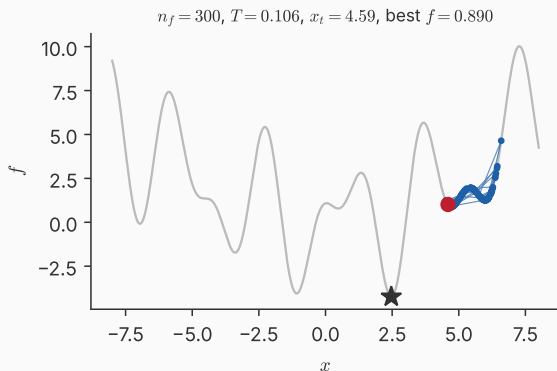
- $n_f = 60$, $T \approx 1.2$: already in the basin of 4.7 (local minimum, $f = 0.89$), but goes up and down a lot.
- $n_f = 150$, $T \approx 0.32$: settled in that basin, with excursions; each step already needs ≈ 30 proposals.

One run step by step: $x_0 = 6$, $T_0 = 3$, $\alpha = 0.8$, $n = 10$, $T_s = 0.05$, $\sigma = 0.5$



- $n_f = 60$, $T \approx 1.2$: already in the basin of 4.7 (local minimum, $f = 0.89$), but goes up and down a lot.
- $n_f = 150$, $T \approx 0.32$: settled in that basin, with excursions; each step already needs ≈ 30 proposals.
- $n_f = 300$, $T \approx 0.11$: excursions rare and short.

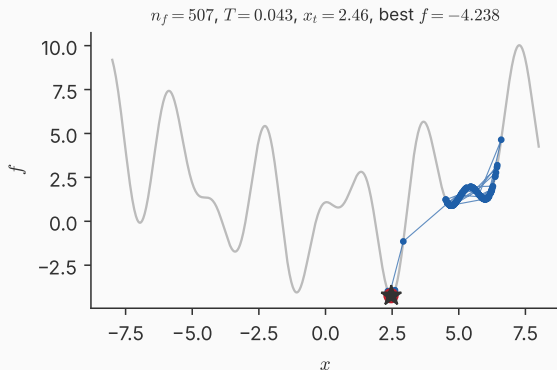
One run step by step: $x_0 = 6$, $T_0 = 3$, $\alpha = 0.8$, $n = 10$, $T_s = 0.05$, $\sigma = 0.5$



- $n_f = 60$, $T \approx 1.2$: already in the basin of 4.7 (local minimum, $f = 0.89$), but goes up and down a lot.
- $n_f = 150$, $T \approx 0.32$: settled in that basin, with excursions; each step already needs ≈ 30 proposals.
- $n_f = 300$, $T \approx 0.11$: excursions rare and short.

Question: do you think it will manage to escape? (*It is stochastic: it may or may not.*)

One run step by step: $x_0 = 6$, $T_0 = 3$, $\alpha = 0.8$, $n = 10$, $T_s = 0.05$, $\sigma = 0.5$

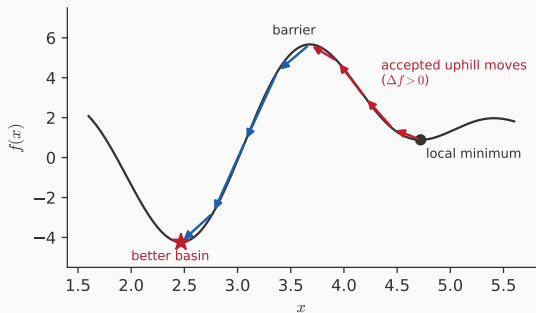


- $n_f = 60$, $T \approx 1.2$: already in the basin of 4.7 (local minimum, $f = 0.89$), but goes up and down a lot.
- $n_f = 150$, $T \approx 0.32$: settled in that basin, with excursions; each step already needs ≈ 30 proposals.
- $n_f = 300$, $T \approx 0.11$: excursions rare and short.

Question: do you think it will manage to escape? (*It is stochastic: it may or may not.*)

At $n_f = 325$ ($T = 0.084$) a proposal lands **on the other side of the barrier**, at a lower point, and is accepted ($4.59 \rightarrow 2.92$). It terminates with $T = 0.043 < T_s$ at $x_t = 2.457$: $n_f = 507$, 191 accepted. A single run (seed 3): **it went well this time.**

How is a barrier crossed?



From the basin of 4.72 to that of 2.47: barrier of height $H \approx 4.8$.

SA does not know where the better basin is. There are two ways to get there:

A sequence of accepted climbs

At roughly constant T , uphill moves with total rise H have an acceptance factor $e^{-H/T}$: 0.20 at $T = 3$, 0.008 at $T = 1$, 6×10^{-6} at $T = 0.4$.

Low T traps.

A jump

A proposal lands directly on the other side, at a lower point — accepted as a descent. This is what happened in the previous run ($n_f = 325$: from 4.59 to 2.92, at 3.3σ). A small σ makes such jumps rare.

More budget: more attempts.

One run is not enough: 100 runs

Success ($|x - x^*| < 0.3$) in 100 runs, always from $x_0 = 6$, with $T_s = 0.05$:

σ	T_0	α	n	n_f [Q1, Q3]	success	
					final x_t	best
0.5	3.0	0.80	10	693 [525, 828]	60%	63%
0.5	3.0	0.50	10	148 [136, 168]	11%	11%
0.5	3.0	0.95	10	3318 [3187, 3470]	69%	93%
0.5	0.3	0.80	10	404 [368, 440]	15%	15%
1.0	3.0	0.80	10	1518 [1445, 1578]	60%	92%
0.5	3.0	0.80	50	3942 [3669, 4125]	64%	93%

With the parameters of the previous run, **60%**. The successful run was one among many possible: **one SA run is not an experimental conclusion.**

Stochastic algorithm $\Rightarrow$ several runs

Report the **success rate**, **median** and **spread** over ≥ 20 –30 runs, with fixed seeds.

With long exploration ($\alpha = 0.95$, $\sigma = 1$, $n = 50$) the final point may end in the basin of -1.08 ($f = -4.05$, almost as low as the global one): **keeping the best visited** pays off.

What do the parameters do?

parameter	too small	too large
T_0	starts almost local ($T_0 = 0.3$: 15%)	accepts almost everything; wanders
σ	does not cross barriers	distant proposals, more rejected ($\sigma = 1$: $2 \times n_f$)
α	cools quickly ($\alpha = 0.5$: 11%)	long exploration ($\alpha = 0.95$: $n_f \approx 3300$)
n	little time at each temperature	higher cost ($n = 50$: $n_f \approx 3900$)
T_s	costly cold steps	stops before refining

T : what we accept

tolerance to getting worse

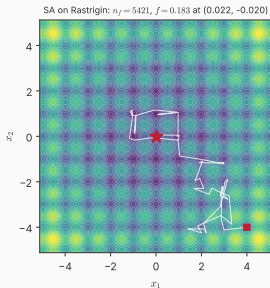
σ : where we propose

geometric exploration, matched to the scale of the basins ($\sigma = 1$: 92% with the best visited)

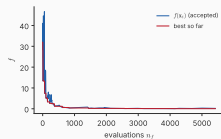
T and σ play different roles; the cooling schedule (T_0, α, n, T_s) balances exploration and refinement.

In two dimensions: Rastrigin

$f(\mathbf{x}) = 20 + \sum_{i=1}^2 (x_i^2 - 10 \cos 2\pi x_i)$ on $[-5.12, 5.12]^2$: 121 local minima on a grid; global $f = 0$ at $\mathbf{0}$.



Here fast cooling also works ($\alpha = 0.3$: 97%); the effect of cooling shows on the 1D function.



- $T_0 = 20$, $\alpha = 0.5$, $n = 10$, $T_s = 0.1$, $\sigma = 0.5$, from $(4, -4)$: terminates at $f = 0.18$ (global basin) with $n_f = 5421$ and **only 81 accepted**: almost all evaluations go to the cold steps.
- 30 runs from random points, success $f < 1$: 90% (best visited: 100%), $n_f \approx 4800$; with $\alpha = 0.8$, 97%, but $n_f \approx 18,600$. On another sample (the one in 5.3): Nelder–Mead **3%** ($n_f \approx 90$), SA 97% ($n_f \approx 5300$).
- Modest final accuracy ($f \sim 10^{-1}$): finish with a local descent (Ch. 3).

The price of simulated annealing

Advantages

- Extremely simple.
- Only values of f .
- Can escape from local minima.
- Adapts to discrete variables and permutations through the neighbourhood.
- One evaluation per proposal.

SA → local method

SA finds a good basin; a local method (Chs. 2–3) refines. In the example: from $x_{\text{SA}} = 2.457$, a direct search reaches $|x - x^*| < 10^{-6}$ with **38 evaluations** (SA used 507).

Limitations

- Many evaluations.
- Random result.
- T_0 , α , n , σ strongly affect performance.
- More effective at exploring than at refining to high accuracy.
- The global guarantee is asymptotic, with impractically slow schedules.

Exploring globally and refining locally are different tasks — and can be done by different methods (the GLODS philosophy, seen another way).

SA does not “know” it has found the global one: it returns the final point (or the best visited), with no certificate of globality.

GLODS and SA: what changed?

	GLODS (5.1)	SA (5.2)
State	several points	one point
How it escapes	the <i>search</i> finds other regions	accepts uphill moves
Local search	direct search (<i>poll</i>)	not explicit (random walk)
Memory	list of minima / searches	current point (+ $\mathbf{x}_{\text{best}}$)
Random?	depends on the <i>search</i>	yes
Natural output	several minima	the final point (or $\mathbf{x}_{\text{best}}$)

It is not “which is better”: they are two different answers to the same question of the chapter.

On the computer: MATLAB implementation

```
% core of SimulatedAnnealing(f, x0, T0, alpha, n, Ts, sigma, lb, ub)
x = x0; fx = f(x); T = T0; t = 0;
while true
    xn = min(max(x + sigma*randn(size(x)), lb), ub); % step 2
    fn = f(xn); df = fn - fx;
    if df < 0 || rand < exp(-df/T) % step 3
        x = xn; fx = fn; t = t + 1;
        if T < Ts, break, end % step 4
        if mod(t, n) == 0, T = alpha*T; end
    end
end
```

The full function (SimulatedAnnealing) also keeps info.xbest, counts info.nfev and has a limit nfmax.

```
f = @(x) 20 + sum(x.^2 - 10*cos(2*pi*x));
for s = 1:30 % 30 runs
    rng(s); x0 = -5.12 + 10.24*rand(1,2);
    [x, fx, info] = SimulatedAnnealing(f, x0, ...
        20, 0.5, 10, 0.1, 0.5, -5.12, 5.12);
    fs(s) = fx; nf(s) = info.nfev;
end
median(fs), prctile(fs, [25 75]), mean(fs < 1)
```

rng(s) makes each run reproducible; the result is reported with median, quartiles and success rate. Projecting onto the bounds (min/max) is just a simple option.

KEY TAKEAWAYS — 5.2

1. **Global exploration:** a single point that sometimes accepts getting worse.
2. T controls the acceptance of uphill moves; σ , the neighbourhood. In class: n accepted at each T , $T = \alpha T$, until $T < T_s$; in practice, keep the best visited.
3. **Information and cost:** only values of f ; one evaluation per proposal, many proposals.
4. **Randomness:** one run is not enough — several runs; success rate, median and spread.
5. **Guarantee:** current point: only asymptotic (finite space, logarithmic cooling); best visited: that of random search (3.2.1). With a finite budget, no certificate of global optimality.

GLODS: several points exploring different basins. **SA:** one point that can move uphill.

What if we explored several regions simultaneously and let the good solutions exchange information?

→ population → 5.3 Genetic algorithms.

References and resources

References Full list: *Course bibliography*, at the end of the course PDF.

- Deb, K., *Optimization for Engineering Design*, 2nd ed., PHI Learning, 2012 — Ch. 6 (the algorithm used in class).
- Kirkpatrick, S., Gelatt Jr., C. D., Vecchi, M. P., *Optimization by simulated annealing*, Science 220(4598):671–680, 1983.
- Hajek, B., *Cooling schedules for optimal annealing*, Mathematics of Operations Research 13(2):311–329, 1988.
- Hillier, F. S., Lieberman, G. J., *Introduction to Operations Research*, 2024 Release, McGraw-Hill, 2024 — metaheuristics.

Resources

- Companion code: `SimulatedAnnealing` (`info.nfev`, `info.xbest`). MATLAB: `rng(s)`; `simulannealbnd` (Global Optimization Toolbox).
- Student projects (internships at Tecmic, real data): Santos, J. R. (2022) and Costa, M. O. (2023), allocation of transport services; Jin, C. (2024), vehicle assignment for Carris, with SA and NSGA-II; Macau, R. O. (2026), driver scheduling, comparing SA, tabu search and greedy. Full references in the *Course bibliography* (reports in Portuguese).